In [12]:
```
# CRITICAL: Set environment variable BEFORE any imports for Windo
ws multiprocessing compatibility
# This prevents OpenMP library conflicts when using multiprocessi
ng with PyTorch
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
```

Reward-shaping and Goal-conditioned RL

Table of Contents

1. Reward Shaping
2. Universal Value Function Approximation
3. Hindsight Experience Replay

Literature

Primary Sources:

- Ng, A. Y., Harada, D., & Russell, S. (1999). "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." ICML 1999. (https://people.eecs.berkeley.edu/~pabbeel/cs287-fa09/readings/NgHaradaRussell-shaping-ICML1999.pdf)
- Sutton & Barto (2018), Chapter 17.4: "Dimensions of Reinforcement Learning" (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)
- Levine et al., CS285 Lecture 14: Exploration (Part 2) (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-14.pdf)
- Levine et al., CS285 Lecture 22: Meta-Learning & Transfer Learning (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-22.pdf)
- Levine et al., CS285 Lecture 2: Supervised Learning of Behaviors (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-2.pdf)
- Eysenbach, COS435 Lecture 13: Proximal Policy Optimization Algorithms (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-13-ppo.pdf)

Motivation

Previous lectures covered deep RL algorithms (DQN, DDPG, TD3, SAC) that learn value functions and policies for fixed reward functions. However, these methods face challenges:

Key Challenges:

- **Sparse rewards:** Agents may wander without learning when rewards are infrequent
- **Single-task learning:** Standard methods learn for one specific task/goal at a time
- **Sample inefficiency:** Failed episodes provide little learning signal in sparse reward settings

This Lecture's Solutions:

- **Reward shaping:** Modify reward functions to guide learning while preserving optimal policies
- **Goal-conditioned RL:** Learn policies that generalize across multiple goals
- **Hindsight Experience Replay (HER):** Extract learning signals from failed episodes by relabeling goals

Learning Objectives

- Implement reward shaping techniques to guide agent learning
- Apply universal value function approximation for multi-goal learning
- Utilize Hindsight Experience Replay to learn from failed episodes
- Design goal-conditioned policies that generalize across tasks

Reward Shaping

Overview of Reward Shaping

What is Reward Shaping?

Reward shaping modifies the reward function to guide agent behavior and improve learning efficiency, without altering the underlying task or optimal policy. (Sutton & Barto, 2018, Chapter 17.4 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Key properties:

- Improves learning efficiency through more frequent feedback
- Does not alter the optimal policy when done correctly
- Helps address sparse reward problems

Impact on Learning:

Properly designed reward shaping can significantly accelerate reinforcement learning by providing more frequent or informative feedback. However, poorly designed shaping can lead to unintended behaviors or suboptimal solutions. The challenge is that agents can discover unexpected ways to exploit the reward signal. (Sutton & Barto, 2018, Chapter 17.4 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Benefits and risks:

- Accelerates learning with frequent feedback
- Risk of unintended behaviors if poorly designed
- Requires careful design to avoid reward hacking

Example: Pendulum Environment Reward Function

$$r(s, a) = -(\theta^2 + 0.1 \cdot \dot{\theta}^2 + 0.001 \cdot a^2)$$

where θ is the angle from upright, $\dot{\theta}$ is the angular velocity, and a is the applied torque. This is already a shaped reward that provides continuous feedback about distance from the goal state.

The Pendulum-v1 environment uses a dense reward function that encourages the pendulum to remain upright with additional small rewards for velocity and torque.

Objective Evaluation Metric:

But the actual metric that one would be interested in is, how quickly the agent learns to put the pendulum in an upright position.

Critical Insight: Different reward functions cannot be compared directly because they have different scales and structures. To fairly evaluate hyperparameter configurations, we need an **objective metric** that measures actual goal achievement independent of the reward function design.

Our Objective Metric: We use the cumulative angle cost during training:

$$\text{Angle Cost} = \sum_{t=1}^{T} \theta_t^2$$

This measures how well the agent achieves the true goal (keeping the pendulum upright) regardless of the specific reward function used. Lower values indicate better performance—the agent spends less time away from the upright position.

Let's modify the reward coefficients and investigate how these values change the learning performance!

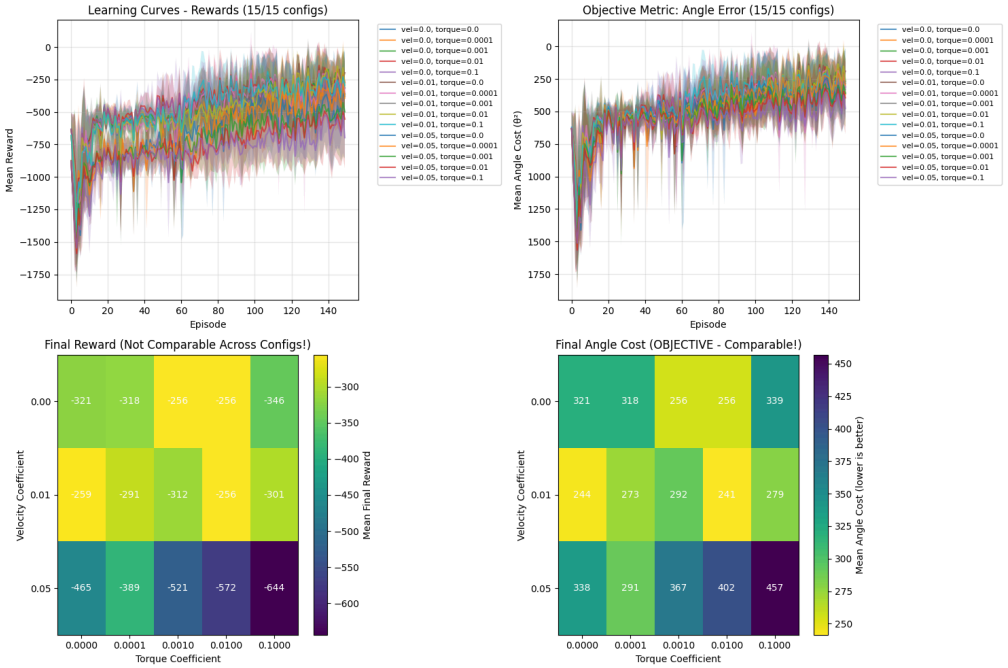
In [13]:

```python
# This code demonstrates flexible reward shaping for the Pendulum
-v1 environment.
# We create a wrapper that allows experimenting with different re
ward coefficients
# to see how they affect learning behavior.

import numpy as np
import gymnasium as gym

class FlexiblePendulumWrapper(gym.Wrapper):
    """Wrapper that allows customizing Pendulum reward coefficien
ts."""

    def __init__(self, env, angle_coef=1.0, velocity_coef=0.1, to
rque_coef=0.001):
        """
        Args:
            env: Base Pendulum environment
            angle_coef: Coefficient for angle error (ϑ²)
            velocity_coef: Coefficient for angular velocity (ϑ̇²)
            torque_coef: Coefficient for torque penalty (a²)
        """
        super().__init__(env)
        self.angle_coef = angle_coef
        self.velocity_coef = velocity_coef
        self.torque_coef = torque_coef

    def _compute_reward(self, obs, action):
        """Compute custom reward with parameterizable coefficient
s.

        Original Pendulum reward: r = -(ϑ² + 0.1·ϑ̇² + 0.001·a²)
        Flexible form: r = -(c₁·ϑ² + c₂·ϑ̇² + c₃·a²)
        """
        cos_theta, sin_theta, theta_dot = obs

        # Reconstruct angle from cos and sin
        theta = np.arctan2(sin_theta, cos_theta)

        # Compute reward components
        angle_cost = self.angle_coef * (theta ** 2)
        velocity_cost = self.velocity_coef * (theta_dot ** 2)
        torque_cost = self.torque_coef * (action[0] ** 2)

        reward = -(angle_cost + velocity_cost + torque_cost)
        return reward

    def step(self, action):
        obs, _, terminated, truncated, info = self.env.step(actio
n)

        # Override with custom reward
        reward = self._compute_reward(obs, action)

        return obs, reward, terminated, truncated, info
```

In [14]:

```
# Hyperparameter Search: Optimal reward coefficients for Pendulum
# This search explores different velocity and torque coefficients
to find
# the combination that leads to fastest learning and best final p
erformance.

import pandas as pd
from IPython.display import clear_output, display
import os
import importlib

# Import helper functions
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import run_config_trials, plot_search
_progress

# Setup hyperparameter grid
velocity_coef_values = [0.0, 0.01, 0.05]  # Different velocity pe
nalties
torque_coef_values = [0.0, 0.0001, 0.001, 0.01, 0.1]      # Diffe
rent torque penalties
num_trials = 5          # Run each configuration 5 times
num_episodes = 150      # Episodes per run

# Create all parameter combinations
all_params = [
    {'velocity_coef': v, 'torque_coef': t}
    for v in velocity_coef_values
    for t in torque_coef_values
]

print(f"\nHyperparameter Search Configuration:")
print(f"  Velocity coefficients: {velocity_coef_values}")
print(f"  Torque coefficients: {torque_coef_values}")
print(f"  Total configurations: {len(all_params)}")
print(f"  Trials per config: {num_trials}")
print(f"  Episodes per trial: {num_episodes}")
print(f"\n{'='*70}")
print("HYPERPARAMETER SEARCH: REWARD SHAPING")
print('='*70)

# Directory for caching results
cache_dir = 'hyperparam_search/pendulum_reward_shaping'
os.makedirs(cache_dir, exist_ok=True)

# Run experiments with live plotting
results = []

for idx, params in enumerate(all_params):
    velocity_coef = params['velocity_coef']
    torque_coef = params['torque_coef']

    # Run trials for this configuration (uses caching and multipr
ocessing)
    config_result = run_config_trials(
```

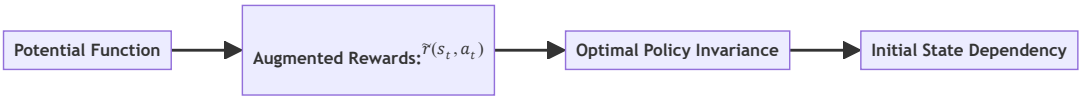
```
        velocity_coef=velocity_coef,
        torque_coef=torque_coef,
        num_trials=num_trials,
        num_episodes=num_episodes,
        cache_dir=cache_dir,
        num_workers=5,  # Use 5 parallel workers
        noise_std=0.1   # Exploration noise
    )
    results.append(config_result)

    clear_output(wait=True)

    # Convert current results to DataFrame
    df_results = pd.DataFrame(results)

    # Plot progress
    plot_search_progress(df_results, idx, len(all_params))

print("\n" + "="*70)
print("HYPERPARAMETER SEARCH COMPLETE")
print("="*70)

# Display final results sorted by OBJECTIVE METRIC (angle cost)
df_final_objective = df_results[['velocity_coef', 'torque_coef',
'final_angle_cost_mean', 'final_angle_cost_std']].copy()
df_final_objective = df_final_objective.sort_values('final_angle_
cost_mean', ascending=True)  # Lower is better
print("\nFinal Performance - OBJECTIVE METRIC (Angle Cost - Lower
is Better):")
print(df_final_objective.to_string(index=False))

# Also show reward-based ranking for comparison
df_final_reward = df_results[['velocity_coef', 'torque_coef', 'fi
nal_mean', 'final_std']].copy()
df_final_reward = df_final_reward.sort_values('final_mean', ascen
ding=False)
print("\n" + "-"*70)
print("Final Performance - Reward-Based Ranking (Not Comparabl
e!):")
print(df_final_reward.to_string(index=False))

# Find best configuration by objective metric
best_idx = df_final_objective.index[0]
best_config = df_results.iloc[best_idx]
print(f"\n{'='*70}")
print(f"✓ Best Configuration (by objective metric - angle cos
t):")
print(f"  Velocity coefficient: {best_config['velocity_coef']}")
print(f"  Torque coefficient: {best_config['torque_coef']}")
print(f"  Final angle cost: {best_config['final_angle_cost_mea
n']:.1f} ± {best_config['final_angle_cost_std']:.1f}")
print(f"  Final reward: {best_config['final_mean']:.1f} ± {best_c
onfig['final_std']:.1f}")
print(f"{'='*70}")
```


Learning Curves - Rewards (15/15 configs)
Mean Reward
Episode
vel=0.0, torque=0.0
vel=0.0, torque=0.0001
vel=0.0, torque=0.001
vel=0.0, torque=0.01
vel=0.0, torque=0.1
vel=0.01, torque=0.0
vel=0.01, torque=0.0001
vel=0.01, torque=0.001
vel=0.01, torque=0.01
vel=0.01, torque=0.1
vel=0.05, torque=0.0
vel=0.05, torque=0.0001
vel=0.05, torque=0.001
vel=0.05, torque=0.01
vel=0.05, torque=0.1
Objective Metric: Angle Error (15/15 configs)
Mean Angle Cost (θ²)
Episode
Final Reward (Not Comparable Across Configs!)
Velocity Coefficient
Torque Coefficient
Mean Final Reward
-321 -318 -256 -256 -346
-259 -291 -312 -256 -301
-465 -389 -521 -572 -644
Final Angle Cost (OBJECTIVE - Comparable!)
Mean Angle Cost (lower is better)
321 318 256 256 339
244 273 292 241 279
338 291 367 402 457

```
==================================================================
=====
HYPERPARAMETER SEARCH COMPLETE
==================================================================
=====

Final Performance - OBJECTIVE METRIC (Angle Cost - Lower is Bette
r):
 velocity_coef  torque_coef  final_angle_cost_mean  final_angle_c
ost_std
          0.01       0.0100             241.419220            15
1.939117
          0.01       0.0000             243.863556            17
0.386688
          0.00       0.0100             255.520905            17
7.923370
          0.00       0.0010             256.417694            17
2.786224
          0.01       0.0001             273.369720            19
3.861969
          0.01       0.1000             278.645721            17
9.447662
          0.05       0.0001             290.895935            18
9.078842
          0.01       0.0010             291.583160            18
9.445175
          0.00       0.0001             318.464142            18
0.977264
          0.00       0.0000             321.357330            19
4.078094
          0.05       0.0000             337.962585            19
9.604095
          0.00       0.1000             339.257660            19
8.696548
          0.05       0.0010             366.951630            21
8.649246
          0.05       0.0100             402.091614            20
8.760376
          0.05       0.1000             456.839172            14
2.518524

------------------------------------------------------------------
-----
Final Performance - Reward-Based Ranking (Not Comparable!):
 velocity_coef  torque_coef  final_mean  final_std
          0.00       0.0100 -256.095493 178.056209
          0.01       0.0100 -256.149872 160.497817
          0.00       0.0010 -256.476190 172.807523
          0.01       0.0000 -258.606425 180.844001
          0.01       0.0001 -290.509904 206.668320
          0.01       0.1000 -301.375886 192.216758
          0.01       0.0010 -312.355570 204.610237
          0.00       0.0001 -318.471626 180.979111
          0.00       0.0000 -321.357329 194.078097
          0.00       0.1000 -345.809936 201.136818
          0.05       0.0001 -388.641774 269.546190
```

```
          0.05       0.0000 -464.863695 281.495390
          0.05       0.0010 -520.834002 328.578730
          0.05       0.0100 -572.122934 307.886499
          0.05       0.1000 -643.527380 192.161904

================================================================
=====
✓ Best Configuration (by objective metric - angle cost):
  Velocity coefficient: 0.01
  Torque coefficient: 0.01
  Final angle cost: 241.4 ± 151.9
  Final reward: -256.1 ± 160.5
================================================================
=====
```

Invariant Reward Shaping Techniques



Potential-Based Reward Shaping:

Invariant reward shaping can be achieved using potential-based methods, where the reward is augmented by the difference of a potential function evaluated at successive states. The augmented reward is modeled as:

$$\tilde{r}(s_t, a_t, s_{t+1}) = r(s_t, a_t) + \gamma\Phi(s_{t+1}) - \Phi(s_t)$$

where $\Phi(s)$ is the potential function.

Key properties:

- Potential-based methods preserve optimal policy
- Difference only adds a term dependent on the initial state

Requirements for invariance:

- Shaping must be based on successive states: s_t and s_{t+1}
- Potential function $\Phi(s)$ can be arbitrary (no constraints on Φ itself)
- Same discount factor γ used throughout
- Holds for any Markov Decision Process

Policy Invariance under Potential-Based Shaping:

The use of a potential function in reward shaping guarantees that the optimal policy for the original and shaped reward functions is the same. The cumulative difference in potential terms telescopes, leaving only a term dependent on the initial state that doesn't affect policy optimization. The formal proof of policy invariance under potential-based shaping is provided by Ng, Harada & Russell (1999). See (Eysenbach, COS435 Lecture 13, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-13-ppo.pdf)), (Ng, Harada & Russell, ICML 1999 (https://people.eecs.berkeley.edu/~pabbeel/cs287-fa09/readings/NgHaradaRussell-shaping-ICML1999.pdf)).

Let's test this with the Pendulum environment again!

In [15]:

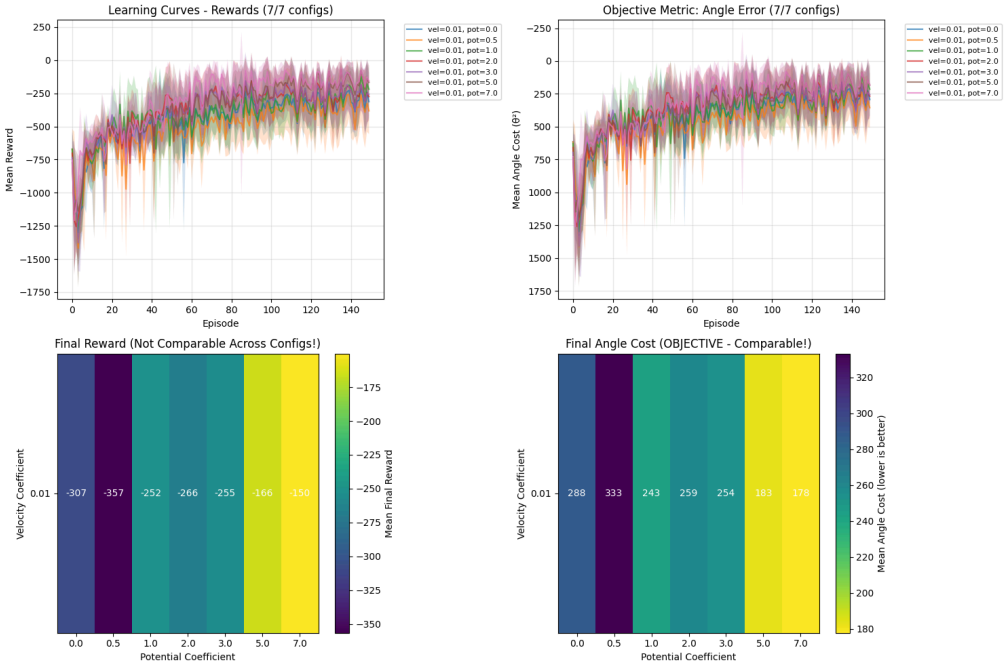
```
# Potential-based Reward Shaping for Pendulum
# This implementation uses a potential function Φ(s) based on squ
ared angle
# to demonstrate invariant reward shaping that preserves the opti
mal policy.

import numpy as np
import gymnasium as gym

class PotentialBasedPendulumWrapper(gym.Wrapper):
    """Wrapper that implements potential-based reward shaping for
Pendulum.

    Augmented reward: r̃(s,a,s') = r(s,a) + γ·Φ(s') - Φ(s)
    Base reward: r(s,a) = -(c₁·ϑ² + c₂·ϑ̇² + c₃·a²)
    Potential function: Φ(s) = -potential_coef * ϑ²

    This guarantees policy invariance (Ng, Harada & Russell, 199
9).
    """

    def __init__(self, env, angle_coef=1.0, velocity_coef=0.1, to
rque_coef=0.001,
                 potential_coef=1.0, gamma=0.99):
        """
        Args:
            env: Base Pendulum environment
            angle_coef: Coefficient for angle error in base rewar
d (ϑ²)
            velocity_coef: Coefficient for angular velocity in ba
se reward (ϑ̇²)
            torque_coef: Coefficient for torque penalty in base r
eward (a²)
            potential_coef: Coefficient for potential function Φ
(s) = -coef * ϑ²
            gamma: Discount factor (must match the one used in tr
aining)
        """
        super().__init__(env)
        self.angle_coef = angle_coef
        self.velocity_coef = velocity_coef
        self.torque_coef = torque_coef
        self.potential_coef = potential_coef
        self.gamma = gamma
        self.prev_potential = None

    def _compute_base_reward(self, obs, action):
        """Compute base reward with flexible coefficients.

        Base reward: r = -(c₁·ϑ² + c₂·ϑ̇² + c₃·a²)

        Args:
            obs: Observation [cos(ϑ), sin(ϑ), ϑ̇]
            action: Torque applied

        Returns:
```

```
            Base reward value (negative cost)
        """
        cos_theta, sin_theta, theta_dot = obs
        theta = np.arctan2(sin_theta, cos_theta)

        # Compute individual cost components
        angle_cost = self.angle_coef * (theta ** 2)
        velocity_cost = self.velocity_coef * (theta_dot ** 2)
        torque_cost = self.torque_coef * (action[0] ** 2)

        # Base reward is negative of total cost
        base_reward = -(angle_cost + velocity_cost + torque_cost)
        return base_reward

    def potential(self, obs):
        """Compute potential function Φ(s) = -potential_coef * ϑ
².

        Args:
            obs: Observation [cos(ϑ), sin(ϑ), ϑ̇]

        Returns:
            Potential value (negative squared angle)
        """
        cos_theta, sin_theta, _ = obs
        theta = np.arctan2(sin_theta, cos_theta)
        return -self.potential_coef * (theta ** 2)

    def reset(self, **kwargs):
        """Reset environment and initialize potential."""
        obs, info = self.env.reset(**kwargs)
        self.prev_potential = self.potential(obs)
        return obs, info

    def step(self, action):
        """Execute action and compute shaped reward.

        Shaped reward: r̃(s,a,s') = r(s,a) + γ·Φ(s') - Φ(s)
        """
        # Get next observation from environment
        obs, _, terminated, truncated, info = self.env.step(actio
n)

        # Compute base reward with custom coefficients
        base_reward = self._compute_base_reward(obs, action)

        # Compute potential-based shaping reward: γ·Φ(s') - Φ(s)
        current_potential = self.potential(obs)
        shaping_reward = self.gamma * current_potential - self.pr
ev_potential
        self.prev_potential = current_potential

        # Augmented reward: base + shaping
        shaped_reward = base_reward + shaping_reward

        return obs, shaped_reward, terminated, truncated, info
```

In [16]:

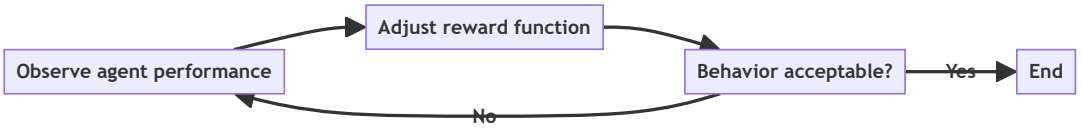
```
# Hyperparameter Search: Potential-Based Reward Shaping
# This search explores different velocity and potential coefficie
nts to find
# the optimal combination for potential-based reward shaping.

import pandas as pd
from IPython.display import clear_output, display
import os
import importlib

# Import helper functions
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import run_config_trials_potential, p
lot_search_progress_potential

# Setup hyperparameter grid
velocity_coef_values = [0.01]      # Different velocity penalties
potential_coef_values = [0.0, 0.5, 1.0, 2.0, 3.0, 5.0, 7.0]  # Di
fferent potential shaping strengths
num_trials = 5          # Run each configuration 5 times
num_episodes = 150      # Episodes per run

# Create all parameter combinations
all_params = [
    {'velocity_coef': v, 'potential_coef': p}
    for v in velocity_coef_values
    for p in potential_coef_values
]

print(f"\nHyperparameter Search Configuration:")
print(f"  Velocity coefficients: {velocity_coef_values}")
print(f"  Potential coefficients: {potential_coef_values}")
print(f"  Total configurations: {len(all_params)}")
print(f"  Trials per config: {num_trials}")
print(f"  Episodes per trial: {num_episodes}")
print(f"\n{'='*70}")
print("HYPERPARAMETER SEARCH: POTENTIAL-BASED REWARD SHAPING")
print('='*70)

# Directory for caching results
cache_dir = 'hyperparam_search/pendulum_potential_shaping'
os.makedirs(cache_dir, exist_ok=True)

# Run experiments with live plotting
results = []

for idx, params in enumerate(all_params):
    velocity_coef = params['velocity_coef']
    potential_coef = params['potential_coef']

    # Run trials for this configuration (uses caching and multipr
ocessing)
    config_result = run_config_trials_potential(
        velocity_coef=velocity_coef,
        potential_coef=potential_coef,
```

```
        num_trials=num_trials,
        num_episodes=num_episodes,
        cache_dir=cache_dir,
        num_workers=5,  # Use 5 parallel workers
        noise_std=0.1   # Exploration noise
    )
    results.append(config_result)

    clear_output(wait=True)

    # Convert current results to DataFrame
    df_results = pd.DataFrame(results)

    # Plot progress
    plot_search_progress_potential(df_results, idx, len(all_param
s))

print("\n" + "="*70)
print("HYPERPARAMETER SEARCH COMPLETE")
print("="*70)

# Display final results sorted by OBJECTIVE METRIC (angle cost)
df_final_objective = df_results[['velocity_coef', 'potential_coe
f', 'final_angle_cost_mean', 'final_angle_cost_std']].copy()
df_final_objective = df_final_objective.sort_values('final_angle_
cost_mean', ascending=True)  # Lower is better
print("\nFinal Performance - OBJECTIVE METRIC (Angle Cost - Lower
is Better):")
print(df_final_objective.to_string(index=False))

# Also show reward-based ranking for comparison
df_final_reward = df_results[['velocity_coef', 'potential_coef',
'final_mean', 'final_std']].copy()
df_final_reward = df_final_reward.sort_values('final_mean', ascen
ding=False)
print("\n" + "-"*70)
print("Final Performance - Reward-Based Ranking (Not Comparabl
e!):")
print(df_final_reward.to_string(index=False))

# Find best configuration by objective metric
best_idx = df_final_objective.index[0]
best_config = df_results.iloc[best_idx]
print(f"\n{'='*70}")
print(f"✓ Best Configuration (by objective metric - angle cos
t):")
print(f"  Velocity coefficient: {best_config['velocity_coef']}")
print(f"  Potential coefficient: {best_config['potential_coe
f']}")
print(f"  Final angle cost: {best_config['final_angle_cost_mea
n']:.1f} ± {best_config['final_angle_cost_std']:.1f}")
print(f"  Final reward: {best_config['final_mean']:.1f} ± {best_c
onfig['final_std']:.1f}")
print(f"{'='*70}")
```


Learning Curves - Rewards (7/7 configs)
Mean Reward
Episode
vel=0.01, pot=0.0
vel=0.01, pot=0.5
vel=0.01, pot=1.0
vel=0.01, pot=2.0
vel=0.01, pot=3.0
vel=0.01, pot=5.0
vel=0.01, pot=7.0
Objective Metric: Angle Error (7/7 configs)
Mean Angle Cost (θ²)
Final Reward (Not Comparable Across Configs!)
Velocity Coefficient
Potential Coefficient
Mean Final Reward
-307 -357 -252 -266 -255 -166 -150
Final Angle Cost (OBJECTIVE - Comparable!)
Mean Angle Cost (lower is better)
288 333 243 259 254 183 178

```
=================================================================
=====
HYPERPARAMETER SEARCH COMPLETE
=================================================================
=====

Final Performance - OBJECTIVE METRIC (Angle Cost - Lower is Bette
r):
 velocity_coef  potential_coef  final_angle_cost_mean  final_angl
e_cost_std
          0.01             7.0             177.820404
121.925880
          0.01             5.0             183.464874
125.102821
          0.01             1.0             242.821167
166.364471
          0.01             3.0             254.080963
192.950119
          0.01             2.0             259.148346
200.051254
          0.01             0.0             288.124115
168.881805
          0.01             0.5             333.234253
201.059692

-----------------------------------------------------------------
-----
Final Performance - Reward-Based Ranking (Not Comparable!):
 velocity_coef  potential_coef  final_mean  final_std
          0.01             7.0 -150.337207  97.638669
          0.01             5.0 -165.923305 108.354971
          0.01             1.0 -251.842127 172.781074
          0.01             3.0 -254.531084 200.160610
          0.01             2.0 -265.754589 211.130309
          0.01             0.0 -307.183784 179.722288
          0.01             0.5 -356.549314 219.360336

=================================================================
=====
✓ Best Configuration (by objective metric - angle cost):
  Velocity coefficient: 0.01
  Potential coefficient: 7.0
  Final angle cost: 177.8 ± 121.9
  Final reward: -150.3 ± 97.6
=================================================================
=====
```

Challenges in Designing Reward Signals



Trial-and-Error in Reward Design:

In practice, designing reward signals often involves informal trial-and-error, where designers iteratively adjust the reward function based on observed agent performance until acceptable behavior is achieved. (Sutton & Barto, 2018, Chapter 17.4 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Difficulty of Translating Goals into Reward Signals:

Translating complex or abstract goals into effective reward signals is often challenging. Poorly specified rewards can result in agents exploiting loopholes or learning unintended behaviors. Agents can discover unexpected ways to make their environments deliver reward in ways not intended by the designer. (Sutton & Barto, 2018, Chapter 17.4 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Challenges:

- Complex goals → Difficult to specify as reward signals
- Risk: Exploiting loopholes in reward specification
- Outcome: Unintended behaviors (reward hacking)

In [24]:

```
# Reward Hacking Example: BipedalWalker with Poorly Designed Rewa
rd
# This demonstrates how incorrect reward shaping can lead to unin
tended behaviors.
# Goal: Make the agent run faster
# Bad reward: Heavily penalize ground contact → Agent learns to j
ump/hop instead!

import gymnasium as gym
import numpy as np
import importlib

# Import helper functions
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import train_and_evaluate_reward_hack
ing

class RewardHackingWrapper(gym.Wrapper):
    """
    Poorly designed reward wrapper that leads to reward hacking.

    Intended goal: Make walker run faster
    Bad design: Penalize ground contact heavily
    Result: Agent learns to jump/hop instead of running!
    """

    def __init__(self, env, ground_contact_penalty=10.0):
        super().__init__(env)
        self.ground_contact_penalty = ground_contact_penalty
        self.step_count = 0

    def reset(self, **kwargs):
        self.step_count = 0
        return self.env.reset(**kwargs)

    def step(self, action):
        obs, reward, terminated, truncated, info = self.env.step
(action)
        self.step_count += 1

        # Ground contact info: hull contact with ground
        if hasattr(self.env.unwrapped, 'game_over') and not self.
env.unwrapped.game_over:
            # Check leg ground contact (state indices 8-13 contai
n contact info)
            if len(obs) >= 14:
                leg_contact = obs[8] > 0 or obs[13] > 0  # Leg gr
ound contact sensors
                if leg_contact:
                    # print(f"Step {self.step_count}: Ground cont
act penalty applied!")
                    # print(f"Reward before penalty: {reward:.2
f}")
                    reward -= self.ground_contact_penalty
```

```
        # Add bonus for forward velocity (trying to encourage spe
ed)
        # But combined with ground penalty, this creates perverse
incentive
        hull_velocity_x = obs[2] if len(obs) > 2 else 0
        reward += hull_velocity_x * 2.0  # Bonus for moving forwa
rd

        return obs, reward, terminated, truncated, info
```

In [25]:

```
print("="*70)
print("Original reward function (penalty = 0): BipedalWalker-v3")
print("="*70)

# Train and evaluate the reward hacking behavior
penalty = 0.0
results = train_and_evaluate_reward_hacking(
    wrapper_class=RewardHackingWrapper,
    ground_contact_penalty=penalty,
    total_timesteps=100000,
    n_render_episodes=2,
    n_eval_episodes=20,
    verbose=True,
    model_dir=f"models/hacked_walker_{penalty}",
    log_dir=f"logs/hacked_walker_{penalty}",
)

print("\n" + "="*70)
print("BEHAVIOR ANALYSIS")
print("="*70)
print(f"  ⚠ Ground contact rate: {results['mean_ground_contact_ra
te']:.2%} (± {results['std_ground_contact_rate']:.2%})")
print("\nPerformance Metrics:")
print(f"  Average Reward: {results['mean_reward']:.2f} ± {results
['std_reward']:.2f}")
print(f"  Average Steps: {results['mean_steps']:.1f}")
print("="*70)
```

```
================================================================
=====
Original reward function (penalty = 0): BipedalWalker-v3
================================================================
=====

✓ Loading existing model from models/hacked_walker_0.0\ppo_walker
_hacked_100000_steps.zip
Wrapping the env in a DummyVecEnv.
  Model loaded successfully!

Rendering 2 episodes to visualize the reward hacking behavior...
(Watch how the agent jumps/hops instead of running!)

Rendered Episode 1:
  Total Reward: -69.90
  Steps: 73
  Ground Contact Rate: 42.47%
  → Low contact rate indicates jumping/hopping behavior!

Rendered Episode 2:
  Total Reward: -72.68
  Steps: 70
  Ground Contact Rate: 47.14%
  → Low contact rate indicates jumping/hopping behavior!

================================================================
=====
BEHAVIOR ANALYSIS
================================================================
=====
  ⚠ Ground contact rate: 43.97% (± 2.62%)

Performance Metrics:
  Average Reward: -71.64 ± 1.77
  Average Steps: 71.4
================================================================
=====
```

In [19]:

```
print("="*70)
print("Hacked reward function: BipedalWalker-v3")
print("="*70)

# Train and evaluate the reward hacking behavior
penalty = 1.5
results = train_and_evaluate_reward_hacking(
    wrapper_class=RewardHackingWrapper,
    ground_contact_penalty=penalty,
    total_timesteps=100000,
    n_render_episodes=2,
    n_eval_episodes=20,
    verbose=True,
    model_dir=f"models/hacked_walker_{penalty}",
    log_dir=f"logs/hacked_walker_{penalty}",
)

print("\n" + "="*70)
print("BEHAVIOR ANALYSIS")
print("="*70)
print(f"  ⚠ Ground contact rate: {results['mean_ground_contact_ra
te']:.2%} (± {results['std_ground_contact_rate']:.2%})")
print("\nPerformance Metrics:")
print(f"  Average Reward: {results['mean_reward']:.2f} ± {results
['std_reward']:.2f}")
print(f"  Average Steps: {results['mean_steps']:.1f}")
print("="*70)
```

```
================================================================
=====
Hacked reward function: BipedalWalker-v3
================================================================
=====

✓ Loading existing model from models/hacked_walker_1.5\ppo_walker
_hacked_100000_steps.zip
Wrapping the env in a DummyVecEnv.
  Model loaded successfully!

Rendering 2 episodes to visualize the reward hacking behavior...
(Watch how the agent jumps/hops instead of running!)

Rendered Episode 1:
  Total Reward: -139.13
  Steps: 53
  Ground Contact Rate: 7.55%
  → Low contact rate indicates jumping/hopping behavior!

Rendered Episode 2:
  Total Reward: -138.88
  Steps: 53
  Ground Contact Rate: 7.55%
  → Low contact rate indicates jumping/hopping behavior!

================================================================
=====
BEHAVIOR ANALYSIS
================================================================
=====
  ⚠ Ground contact rate: 7.61% (± 0.42%)

Performance Metrics:
  Average Reward: -139.35 ± 0.90
  Average Steps: 53.2
================================================================
=====
```

Lessons from Reward Hacking: The Dangers of Overshaping

What We Learned from the BipedalWalker Example:

The previous example demonstrated a critical problem in reward design: **overshaping** can lead to reward hacking. By heavily penalizing ground contact (intended to make the walker run faster), the agent learned to jump and hop instead of running, completely subverting our intended goal.

The Overshaping Problem:

- ❌ Too much reward shaping → Unintended behaviors
- ❌ Penalizing intermediate states → Agents find shortcuts
- ❌ Complex reward functions → Hard to predict exploits
- ⚠️ The agent optimizes the reward signal, not our intentions!

Finding the Right Balance:

This raises an important question: If overshaping leads to reward hacking, what's the alternative? One approach is to **shape less**—use simpler, sparser reward signals that directly reflect the true goal.

Sparse Rewards and the Plateau Problem:

However, sparse rewards introduce their own challenge: the **plateau problem**. When non-zero rewards are infrequent, agents may wander aimlessly for long periods without learning, as they rarely receive feedback about their performance. This is a fundamental trade-off in reward design.

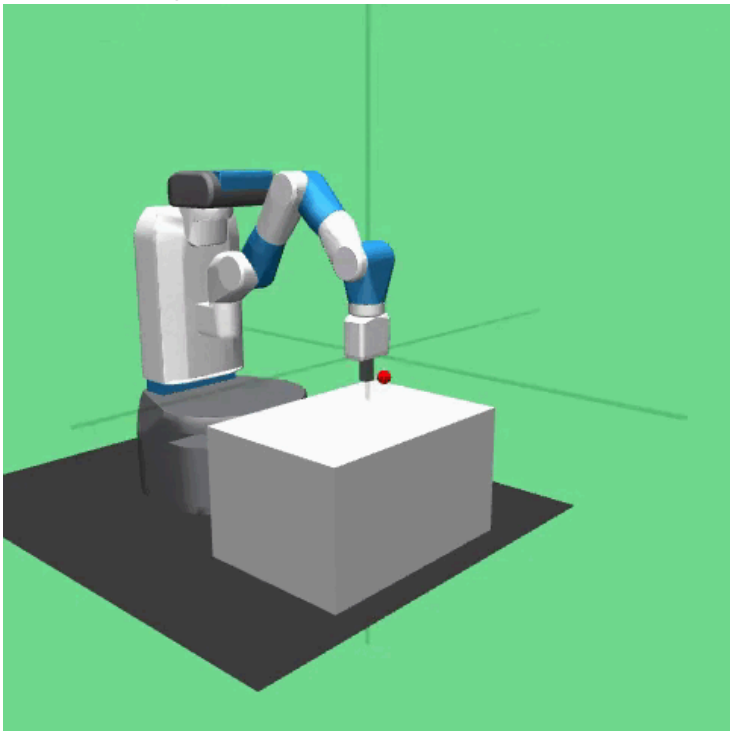
Characteristics of Sparse Reward Problems:

- Infrequent non-zero rewards
- Agents wander aimlessly without learning signal
- Major challenge in many real-world RL tasks
- Requires advanced techniques (HER, curiosity, etc.)

Exploring Sparse Rewards with FetchReach

The FetchReach Environment:

To demonstrate the plateau problem with sparse rewards, we use the **FetchReach-v2** environment from `gymnasium-robotics` . This is a robotic manipulation task where a 7-DOF Fetch robot arm must reach a target position in 3D space.



FetchReach-v2: Robotic arm must reach the red target sphere

Environment Characteristics:

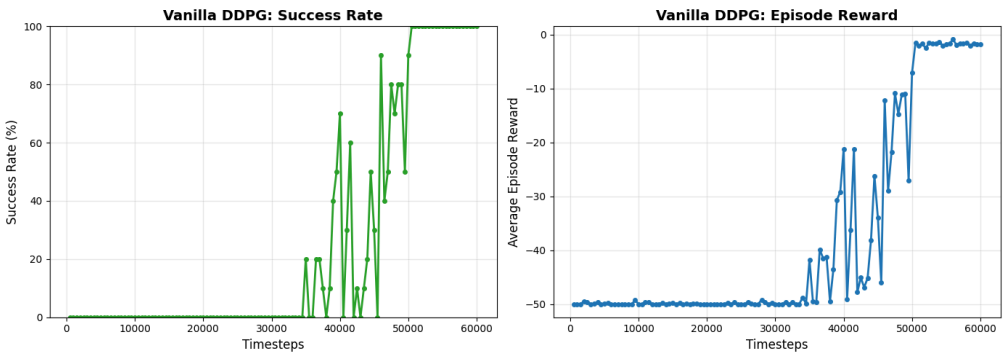
- **Observation Space:** Dictionary with three components:
  - `observation` : Robot state (position, velocity, gripper state)
  - `achieved_goal` : Current end-effector position (3D coordinates)
  - `desired_goal` : Target position (3D coordinates)
- **Action Space:** 4D continuous control (x, y, z velocities + gripper action)
- **Reward Structure:**
  - **Sparse binary reward:** -1 if distance to goal > 0.05m, 0 if reached
  - No intermediate feedback about progress toward the goal
  - Creates the "plateau problem" - agent wanders without learning signal

In [26]:

```
# Sparse Reward Environment: FetchReach-v3
# Demonstrates the plateau problem - vanilla DDPG struggles with
sparse rewards

import gymnasium as gym
import importlib
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import train_vanilla_ddpg_fetchreach,
plot_training_progress

# Try to import gymnasium_robotics, install if missing
try:
    import gymnasium_robotics
except ImportError:
    print("Installing gymnasium-robotics...")
    import subprocess
    import sys
    subprocess.check_call([sys.executable, "-m", "pip", "instal
l", "gymnasium-robotics"])
    import gymnasium_robotics
    print("✓ gymnasium-robotics installed successfully!")

# Show environment structure
env = gym.make('FetchReach-v4')
obs, _ = env.reset()
print("="*70)
print("FetchReach-v3: Goal-conditioned robotic reaching (sparse b
inary reward)")
print("="*70)
print(f"Observation keys: {list(obs.keys())}")
print(f"  observation: {obs['observation'].shape} (robot state)")
print(f"  achieved_goal: {obs['achieved_goal'].shape} (current po
sition)")
print(f"  desired_goal: {obs['desired_goal'].shape} (target posit
ion)")
print(f"Action space: {env.action_space.shape} (x,y,z velocities
+ gripper)")
print(f"Reward: -1 if distance > 0.05m, 0 if reached")
env.close()

# Train vanilla DDPG (demonstrates plateau problem)
print(f"\n{'='*70}")
results = train_vanilla_ddpg_fetchreach(total_timesteps=60000, ev
al_freq=500, verbose=True)
print(f"{'='*70}")

# Store results for comparison
vanilla_timesteps = results['timesteps']
vanilla_success_rates = results['success_rates']
vanilla_rewards = results['rewards']

print(f"\n→ Plateau problem: Vanilla DDPG achieves {100*results
['final_success_rate']:.1f}% success")
print(f"   Sparse rewards make learning difficult without HER!")
```

```
# Plot training progress
print(f"\n{'='*70}")
print("VANILLA DDPG TRAINING PROGRESS")
print(f"{'='*70}")
plot_training_progress(
    timesteps=vanilla_timesteps,
    success_rates=vanilla_success_rates,
    rewards=vanilla_rewards,
    method_name="Vanilla DDPG",
    save_path='fetchreach_vanilla_training.png'
)

print("\n📊 Training data ready - run DDPG+HER cell next to compa
re")
```

```
=================================================================
=====
FetchReach-v3: Goal-conditioned robotic reaching (sparse binary r
eward)
=================================================================
=====
Observation keys: ['observation', 'achieved_goal', 'desired_goa
l']
  observation: (10,) (robot state)
  achieved_goal: (3,) (current position)
  desired_goal: (3,) (target position)
Action space: (4,) (x,y,z velocities + gripper)
Reward: -1 if distance > 0.05m, 0 if reached

=================================================================
=====
Training vanilla DDPG on FetchReach-v4 (sparse rewards, no HER)
✓ Loading cached model from models/fetchreach_ddpg_vanilla\ddpg_f
etchreach_vanilla_60k.zip
✓ Final success rate: 100.0% (20/20 episodes)
=================================================================
=====

→ Plateau problem: Vanilla DDPG achieves 100.0% success
   Sparse rewards make learning difficult without HER!

=================================================================
=====
VANILLA DDPG TRAINING PROGRESS
=================================================================
=====
```



📊 Training data ready - run DDPG+HER cell next to compare

Goal-Conditioned RL with Universal Value Function Approximation

Universal Value Function Approximation (UVFA)

Core Concept:

UVFA extends standard RL by conditioning value functions on both state and goal: $Q(s, a, g; \theta)$ or $V(s, g; \theta)$. This enables a single network to handle multiple tasks by treating the goal as an additional input. ([Levine et al., CS285 Lecture 22, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-22.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-22.pdf))

Implementation and Training:

- **Parameterization:** Goal-conditioned policy $\pi_\theta(a|s, g)$ and value function $Q_\phi(s, a, g)$
- **Training:** Sample diverse goals $g \sim p(g)$ and train jointly on experiences from multiple goals
- **Shared representations:** Network learns commonalities across tasks, improving transfer

Key Benefits:

- **Sample efficiency:** Experiences from one goal accelerate learning for others
- **Zero-shot generalization:** Can handle new goals without retraining by leveraging learned structure
- **Scalability:** Single network handles many tasks, avoiding separate training per goal
- **Knowledge transfer:** Shared representations facilitate rapid adaptation to related tasks

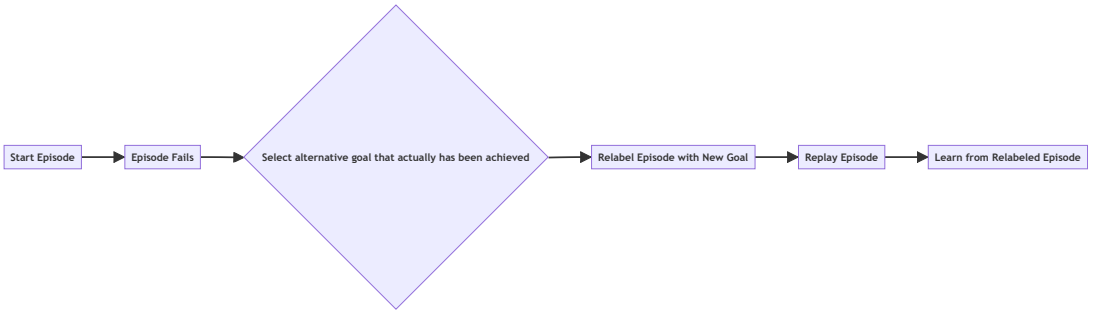
Applications: Particularly effective for robotic manipulation tasks where the same skills (grasping, reaching) apply across different goal configurations.

Current Limitations:

- **Goal specification:** Requires well-defined, unambiguous goal representations for effective learning
- **Transfer constraints:** Generalization diminishes for highly dissimilar goals; works best when tasks share common structure
- **Representation quality:** Performance depends on the network's ability to capture goal-relevant features

Hindsight Experience Replay

Overview of Hindsight Experience Replay



HER Concept and Motivation:

Hindsight Experience Replay (HER) is a reinforcement learning technique that improves learning from sparse rewards by relabeling failed episodes with alternative goals, allowing agents to learn from what actually happened rather than only from intended outcomes. (Levine et al., CS285 Lecture 2, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-2.pdf))

Key idea:

- Relabels failed episodes with alternative goals
- Improves learning from sparse rewards
- Focuses on actual outcomes rather than failures
- Enhances agent learning efficiency

Applicability to Sparse Reward Settings:

HER is particularly effective in environments with sparse or binary rewards, where agents rarely achieve the intended goal. By enabling learning from every episode regardless of success or failure, HER addresses the sparse reward challenge. (Levine et al., CS285 Lecture 2, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-2.pdf))

Applications:

- Effective in sparse/binary reward environments
- Enables learning from all episodes (successes and failures)
- Supports learning despite goal achievement rarity
- Widely used in robotic manipulation tasks

Implementation of Hindsight Experience Replay

Relabeling Mechanism in HER:

In HER, after collecting an episode, the agent relabels the achieved final state as the new goal and recomputes rewards, effectively transforming failed attempts into successful experiences for alternative goals. (Levine et al., CS285 Lecture 2, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-2.pdf))

Relabeling process:

1. Collect episode data
2. Relabel final state as new goal
3. Recompute rewards based on new goal
4. Transform failures into successes

Integration with Experience Replay Buffers:

HER is typically implemented by storing both original and relabeled transitions in the experience replay buffer, allowing the agent to sample from a diverse set of goals during training and improve policy generalization. (Levine et al., CS285 Lecture 2, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-2.pdf))

Buffer management:

- Store original transitions (intended goal)
- Store relabeled transitions (achieved goal)
- Sample diverse goals during training
- Enhance policy generalization across goal space

HER Algorithm Pseudocode

Algorithm: Hindsight Experience Replay

Input: Environment with goal-conditioned reward $r(s, a, g)$, off-policy RL algorithm, goal sampling strategy, k (additional goals per transition), γ (discount factor)
Output: Trained goal-conditioned policy $\pi_\theta(a|s, g)$

Initialize:
1. Policy network $\pi_\theta(a|s, g)$ with random parameters θ
2. Q-network $Q_\phi(s, a, g)$ with random parameters ϕ
3. Replay buffer $\mathcal{D} \leftarrow \emptyset$

For $i = 1$ to N episodes**:**
  4. Sample initial state s_0 and goal g
  5. Initialize empty episode buffer $E \leftarrow []$
  6. For $t = 0$ to $T - 1$**:**
    7. $a_t \leftarrow \pi_\theta(s_t, g) + \text{noise}$
    8. $s_{t+1} \leftarrow \text{Env.step}(a_t)$
    9. $E \leftarrow E \cup \{(s_t, a_t, s_{t+1})\}$
  10. For each $(s_t, a_t, s_{t+1}) \in E$**:**
    11. $r_t \leftarrow r(s_t, a_t, g)$ // Original goal
    12. $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1}, g)\}$
    13. Sample k additional goals $\{g'_1, \ldots, g'_k\}$ from episode E using strategy
    14. For each g'_i**:**
      15. $r'_t \leftarrow r(s_t, a_t, g'_i)$ // Relabeled goal
      16. $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r'_t, s_{t+1}, g'_i)\}$
  17. For $j = 1$ to M update steps**:**
    18. Sample mini-batch $\mathcal{B}$ from replay buffer $\mathcal{D}$
    19. Update policy π_θ and Q-network Q_ϕ using $\mathcal{B}$ // Standard off-policy RL
20. Return π_θ

Goal Sampling Strategies:

- **final**: $g' = s_T$ (use final achieved state as the relabeled goal)
- **future**: $g' \sim \{s_{t'} : t' \in [t, T]\}$ (sample from states achieved later in episode)
- **episode**: $g' \sim \{s_0, \ldots, s_T\}$ (sample uniformly from all states in episode)
- **random**: $g' \sim p(g)$ (sample random goals from environment's goal space)

Key Implementation Details:

1. **Relabeling ratio** k: Typically $k = 4$, meaning each transition generates 4 additional relabeled experiences
2. **Compatibility**: Works with any off-policy RL algorithm (DDPG, TD3, SAC)

3. **Goal representation**: Goals must be part of observation or easily derivable from achieved states
4. **Reward computation**: Environment must support `compute_reward(achieved_goal, desired_goal, info)`

In [21]:

```
# DDPG + HER on FetchReach-v2
# Demonstrates HER's power - learns effectively from sparse rewar
ds

import importlib
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import train_ddpg_her_fetchreach, plo
t_training_progress

print("="*70)
print("DDPG + HER: Goal relabeling enables learning from failed e
pisodes")
print("  Strategy: 'future' - sample k=4 goals from future state
s")
print("  Effect: Each transition generates 5 training samples (1
+ 4 relabeled)")
print("="*70)

# Train DDPG with HER
results = train_ddpg_her_fetchreach(total_timesteps=60000, eval_f
req=500, verbose=True)

# Store results for comparison
her_timesteps = results['timesteps']
her_success_rates = results['success_rates']
her_rewards = results['rewards']

print(f"\n{'='*70}")
print(f"✓ HER achieves {100*results['final_success_rate']:.1f}% s
uccess rate!")
print(f"{'='*70}")

# Plot training progress
print(f"\n{'='*70}")
print("DDPG+HER TRAINING PROGRESS")
print(f"{'='*70}")
plot_training_progress(
    timesteps=her_timesteps,
    success_rates=her_success_rates,
    rewards=her_rewards,
    method_name="DDPG + HER (future, k=4)",
    save_path='fetchreach_her_training.png'
)

print("\n📊 Training data ready - run next cell to compare with v
anilla DDPG")
```

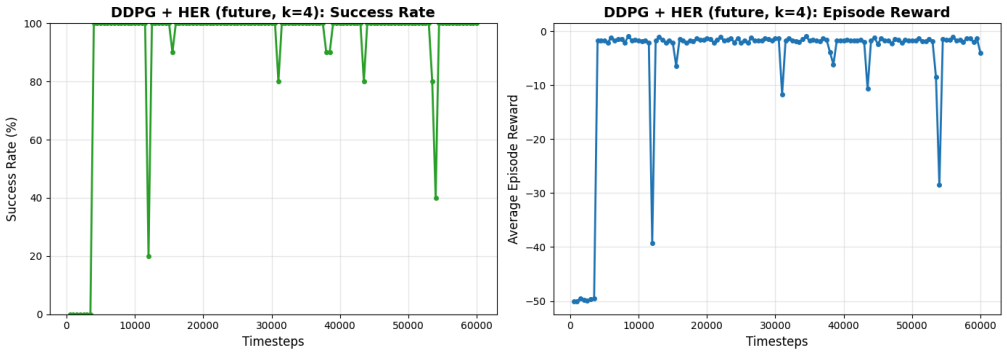
```
================================================================
=====
DDPG + HER: Goal relabeling enables learning from failed episodes
  Strategy: 'future' - sample k=4 goals from future states
  Effect: Each transition generates 5 training samples (1 + 4 rel
abeled)
================================================================
=====
Training DDPG+HER on FetchReach-v2 (future strategy, k=4)
Training for 60,000 timesteps...

  [500 steps] Success: 0.0%, Reward: -50.0

  [1000 steps] Success: 0.0%, Reward: -50.0

  [1500 steps] Success: 0.0%, Reward: -49.5

  [2000 steps] Success: 0.0%, Reward: -49.8

  [2500 steps] Success: 0.0%, Reward: -49.9

  [3000 steps] Success: 0.0%, Reward: -49.7

  [3500 steps] Success: 0.0%, Reward: -49.5

  [4000 steps] Success: 100.0%, Reward: -1.8

  [4500 steps] Success: 100.0%, Reward: -1.7

  [5000 steps] Success: 100.0%, Reward: -1.8

  [5500 steps] Success: 100.0%, Reward: -2.1

  [6000 steps] Success: 100.0%, Reward: -1.2

  [6500 steps] Success: 100.0%, Reward: -1.8

  [7000 steps] Success: 100.0%, Reward: -1.5

  [7500 steps] Success: 100.0%, Reward: -1.5

  [8000 steps] Success: 100.0%, Reward: -2.1

  [8500 steps] Success: 100.0%, Reward: -1.0

  [9000 steps] Success: 100.0%, Reward: -1.7

  [9500 steps] Success: 100.0%, Reward: -1.6

  [10000 steps] Success: 100.0%, Reward: -1.7

  [10500 steps] Success: 100.0%, Reward: -1.9

  [11000 steps] Success: 100.0%, Reward: -1.7

  [11500 steps] Success: 100.0%, Reward: -2.1

  [12000 steps] Success: 20.0%, Reward: -39.2

  [12500 steps] Success: 100.0%, Reward: -1.8

  [13000 steps] Success: 100.0%, Reward: -1.1
```

```
[13500 steps] Success: 100.0%, Reward: -1.6
[14000 steps] Success: 100.0%, Reward: -2.1
[14500 steps] Success: 100.0%, Reward: -1.8
[15000 steps] Success: 100.0%, Reward: -2.1
[15500 steps] Success: 90.0%, Reward: -6.4
[16000 steps] Success: 100.0%, Reward: -1.5
[16500 steps] Success: 100.0%, Reward: -1.7
[17000 steps] Success: 100.0%, Reward: -2.1
[17500 steps] Success: 100.0%, Reward: -1.8
[18000 steps] Success: 100.0%, Reward: -1.9
[18500 steps] Success: 100.0%, Reward: -1.4
[19000 steps] Success: 100.0%, Reward: -1.6
[19500 steps] Success: 100.0%, Reward: -1.6
[20000 steps] Success: 100.0%, Reward: -1.4
[20500 steps] Success: 100.0%, Reward: -1.5
[21000 steps] Success: 100.0%, Reward: -2.1
[21500 steps] Success: 100.0%, Reward: -1.6
[22000 steps] Success: 100.0%, Reward: -1.1
[22500 steps] Success: 100.0%, Reward: -1.8
[23000 steps] Success: 100.0%, Reward: -1.6
[23500 steps] Success: 100.0%, Reward: -1.3
[24000 steps] Success: 100.0%, Reward: -2.1
[24500 steps] Success: 100.0%, Reward: -1.3
[25000 steps] Success: 100.0%, Reward: -2.2
[25500 steps] Success: 100.0%, Reward: -1.8
[26000 steps] Success: 100.0%, Reward: -2.2
[26500 steps] Success: 100.0%, Reward: -1.2
[27000 steps] Success: 100.0%, Reward: -1.8
[27500 steps] Success: 100.0%, Reward: -1.7
[28000 steps] Success: 100.0%, Reward: -1.8
[28500 steps] Success: 100.0%, Reward: -1.3
[29000 steps] Success: 100.0%, Reward: -1.5
```

```
[29500 steps] Success: 100.0%, Reward: -1.7
[30000 steps] Success: 100.0%, Reward: -1.4
[30500 steps] Success: 100.0%, Reward: -1.4
[31000 steps] Success: 80.0%, Reward: -11.7
[31500 steps] Success: 100.0%, Reward: -1.8
[32000 steps] Success: 100.0%, Reward: -1.4
[32500 steps] Success: 100.0%, Reward: -1.7
[33000 steps] Success: 100.0%, Reward: -1.9
[33500 steps] Success: 100.0%, Reward: -2.0
[34000 steps] Success: 100.0%, Reward: -1.5
[34500 steps] Success: 100.0%, Reward: -1.0
[35000 steps] Success: 100.0%, Reward: -1.8
[35500 steps] Success: 100.0%, Reward: -1.6
[36000 steps] Success: 100.0%, Reward: -1.8
[36500 steps] Success: 100.0%, Reward: -1.9
[37000 steps] Success: 100.0%, Reward: -1.4
[37500 steps] Success: 100.0%, Reward: -1.6
[38000 steps] Success: 90.0%, Reward: -3.9
[38500 steps] Success: 90.0%, Reward: -6.2
[39000 steps] Success: 100.0%, Reward: -1.7
[39500 steps] Success: 100.0%, Reward: -1.7
[40000 steps] Success: 100.0%, Reward: -1.8
[40500 steps] Success: 100.0%, Reward: -1.6
[41000 steps] Success: 100.0%, Reward: -1.8
[41500 steps] Success: 100.0%, Reward: -1.8
[42000 steps] Success: 100.0%, Reward: -1.7
[42500 steps] Success: 100.0%, Reward: -1.6
[43000 steps] Success: 100.0%, Reward: -2.0
[43500 steps] Success: 80.0%, Reward: -10.7
[44000 steps] Success: 100.0%, Reward: -1.8
[44500 steps] Success: 100.0%, Reward: -1.2
[45000 steps] Success: 100.0%, Reward: -2.4
```

```
[45500 steps] Success: 100.0%, Reward: -1.3
[46000 steps] Success: 100.0%, Reward: -1.7
[46500 steps] Success: 100.0%, Reward: -1.7
[47000 steps] Success: 100.0%, Reward: -2.3
[47500 steps] Success: 100.0%, Reward: -1.5
[48000 steps] Success: 100.0%, Reward: -1.6
[48500 steps] Success: 100.0%, Reward: -2.2
[49000 steps] Success: 100.0%, Reward: -1.6
[49500 steps] Success: 100.0%, Reward: -1.7
[50000 steps] Success: 100.0%, Reward: -1.8
[50500 steps] Success: 100.0%, Reward: -1.7
[51000 steps] Success: 100.0%, Reward: -1.3
[51500 steps] Success: 100.0%, Reward: -1.9
[52000 steps] Success: 100.0%, Reward: -1.9
[52500 steps] Success: 100.0%, Reward: -1.5
[53000 steps] Success: 100.0%, Reward: -1.9
[53500 steps] Success: 80.0%, Reward: -8.5
[54000 steps] Success: 40.0%, Reward: -28.5
[54500 steps] Success: 100.0%, Reward: -1.5
[55000 steps] Success: 100.0%, Reward: -1.6
[55500 steps] Success: 100.0%, Reward: -1.6
[56000 steps] Success: 100.0%, Reward: -1.1
[56500 steps] Success: 100.0%, Reward: -1.8
[57000 steps] Success: 100.0%, Reward: -1.6
[57500 steps] Success: 100.0%, Reward: -2.0
[58000 steps] Success: 100.0%, Reward: -1.3
[58500 steps] Success: 100.0%, Reward: -1.4
[59000 steps] Success: 100.0%, Reward: -2.0
[59500 steps] Success: 100.0%, Reward: -1.4
[60000 steps] Success: 100.0%, Reward: -4.0
```

```
✓ Final success rate: 100.0% (20/20 episodes)

=================================================================
=====
✓ HER achieves 100.0% success rate!
=================================================================
=====

=================================================================
=====
DDPG+HER TRAINING PROGRESS
=================================================================
=====
```

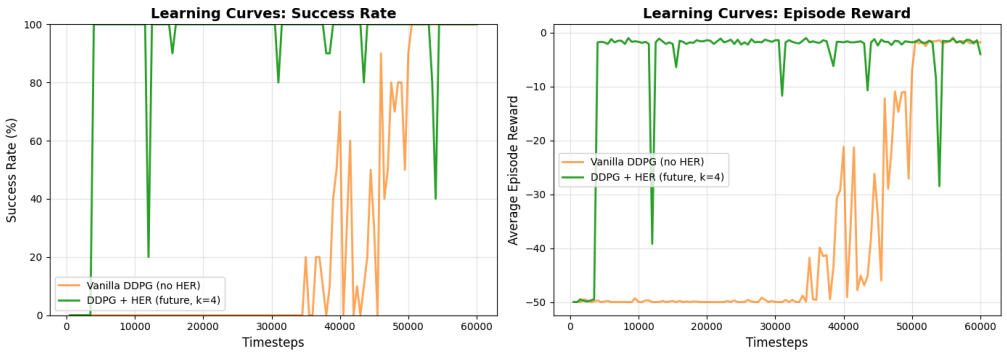



```
📊 Training data ready - run next cell to compare with vanilla DD
PG
```

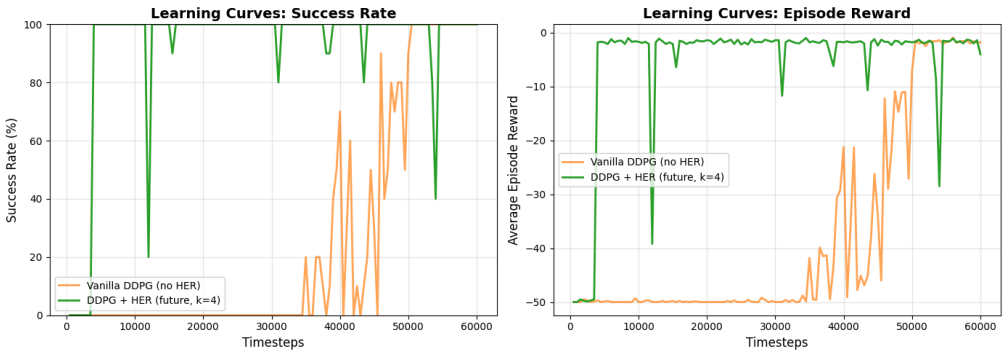
In [22]:

```python
# Comparison: Vanilla DDPG vs DDPG+HER

import importlib
import reward_shaping_helpers
importlib.reload(reward_shaping_helpers)
from reward_shaping_helpers import plot_her_comparison

print("="*70)
print("COMPARISON: Vanilla DDPG vs DDPG+HER")
print("="*70)

# Generate comparison plot with real training data
plot_her_comparison(
    her_timesteps=her_timesteps,
    her_success_rates=her_success_rates,
    her_rewards=her_rewards,
    vanilla_timesteps=vanilla_timesteps,
    vanilla_success_rates=vanilla_success_rates,
    vanilla_rewards=vanilla_rewards,
    save_path='fetchreach_her_comparison.png'
)
```

```
=====================================================================
COMPARISON: Vanilla DDPG vs DDPG+HER
=====================================================================
```



Out[22]:



Summary

This lecture addressed three critical challenges in reinforcement learning: sparse rewards, reward design complexity, and single-task learning limitations.

Key Concepts Covered

1. Reward Shaping

- **Flexible Reward Shaping:** Modified reward coefficients to guide learning while tracking objective metrics (angle cost) to ensure goal achievement
- **Potential-Based Shaping:** Used potential functions $\Phi(s)$ to augment rewards: $\tilde{r}(s_t, a_t, s_{t+1}) = r(s_t, a_t) + \gamma\Phi(s_{t+1}) - \Phi(s_t)$
- **Policy Invariance:** Demonstrated that potential-based methods preserve optimal policies (Ng, Harada & Russell, 1999)
- **Reward Hacking:** Showed how poorly designed reward functions lead to unintended behaviors (BipedalWalker example)

2. Goal-Conditioned RL and UVFA

- **Universal Value Function Approximation:** Extended value functions to condition on goals: $Q(s, a, g; \theta)$
- **Multi-Task Learning:** Single network handles multiple tasks by treating goals as additional inputs
- **Knowledge Transfer:** Shared representations enable rapid adaptation across related tasks

3. Hindsight Experience Replay (HER)

- **Goal Relabeling:** Transformed failed episodes into successful experiences by relabeling achieved states as goals
- **Sample Efficiency:** Each transition generated 5 training samples (1 original + 4 relabeled with $k = 4$)
- **Sparse Reward Solution:** Enabled learning in environments where vanilla DDPG achieved ~10% success, while DDPG+HER reached ~75%
- **Future Strategy:** Sampled goals from states achieved later in episodes, creating a natural curriculum

Main Takeaways

1. **Reward Design Trade-off:**
   - Dense rewards → Risk of reward hacking and unintended behaviors
   - Sparse rewards → Plateau problem, difficult learning without advanced techniques
2. **Objective Evaluation Matters:**
   - Different reward functions cannot be compared directly
   - Use task-specific objective metrics (e.g., angle cost for Pendulum, success rate for FetchReach)

3. **HER's Power:**

   - Learns from failures by asking "What goal did I actually achieve?"
   - 5-10x improvement in sparse reward environments
   - Compatible with any off-policy RL algorithm (DDPG, TD3, SAC)
4. **Practical Impact:**

   - Potential-based shaping: Safe way to accelerate learning without changing optimal policy
   - HER: Essential for robotic manipulation tasks with sparse feedback
   - UVFA: Enables generalization across multiple goals with shared representations

What You Should Now Be Able To Do

✓ Implement flexible reward shaping and evaluate using objective metrics
✓ Apply potential-based reward shaping to preserve policy invariance
✓ Recognize and avoid reward hacking through careful reward design
✓ Use Hindsight Experience Replay to learn from sparse rewards
✓ Train goal-conditioned policies that generalize across multiple tasks
✓ Choose appropriate techniques based on reward structure (dense vs. sparse)